

Complete DBT Notes for Data Engineering

1. Introduction to DBT

What is DBT?

DBT (Data Build Tool) is a transformation tool used in Data Engineering and Analytics Engineering.

DBT helps transform raw data already stored inside a database or data warehouse using SQL.

DBT mainly works in the transformation layer of ELT architecture.

2. Why We Use DBT

Before DBT:

- Data transformations were written manually
- SQL logic was difficult to maintain
- No proper testing
- No lineage tracking
- No modular development

DBT solves these problems by providing:

- Modular SQL development
 - Data testing
 - Documentation
 - DAG visualization
 - Reusable transformations
 - Incremental loading
 - Snapshots
 - Version control support
-

3. ETL vs ELT

ETL (Extract Transform Load)

In ETL: 1. Data extracted from source 2. Data transformed outside database 3. Clean data loaded into warehouse

Example: Python performs transformation before loading into database.

Architecture

Source → Python/Spark → Database

ELT (Extract Load Transform)

In ELT: 1. Raw data extracted 2. Raw data loaded into warehouse first 3. Transformations performed inside warehouse

DBT follows ELT architecture.

Architecture

Source → Database/Warehouse → DBT Transformations

4. Where DBT Works in Pipeline

Real-World Pipeline

CSV/API/Application ↓ Data Ingestion Tool ↓ Database / Data Warehouse ↓ DBT Transformations ↓
Analytics Tables ↓ Dashboard / Reporting

DBT works only in transformation layer.

DBT does NOT:

- ingest data
- store raw data
- replace database

DBT only transforms existing data.

5. DBT Project Structure

```
employee_project/
```

```
|— analyses/  
|— macros/
```

```
|— models/  
|— seeds/  
|— snapshots/  
|— tests/  
|— target/  
|— logs/  
|— dbt_project.yml  
|— packages.yml
```

6. models Folder

This is the most important folder.

Contains SQL transformation files.

Example:

```
SELECT *  
FROM employee_raw_data
```

Every SQL file inside models becomes:

- view
- table
- incremental table
- ephemeral model

based on materialization.

7. Materializations in DBT

What is Materialization?

Materialization defines how DBT stores model results in database.

Types of Materializations

Materialization	Description
view	Creates SQL view
table	Creates physical table
incremental	Loads only new data
ephemeral	Temporary CTE model
materialized_view	Database materialized view

8. View Materialization

Syntax

```
{{ config(materialized='view') }}
```

Features

- Stores query only
- No physical data stored
- Always fetches latest data
- Faster storage
- Slower query execution

Use Cases

- staging models
- lightweight transformations

9. Table Materialization

Syntax

```
{{ config(materialized='table') }}
```

Features

- Stores physical data
- Faster querying
- Uses more storage
- Rebuilt every run

Use Cases

- final models
 - analytical tables
 - reporting tables
-

10. Incremental Materialization

Syntax

```
{{ config(materialized='incremental') }}

SELECT *
FROM source_table

{% if is_incremental() %}

WHERE employee_id >
(
    SELECT MAX(employee_id)
    FROM {{ this }}
)

{% endif %}
```

Purpose

Loads only new records instead of full table.

Benefits

- Faster execution
- Saves compute

- Efficient for large tables
-

Important Terms

is_incremental()

Checks whether model already exists.

{{ this }}

Represents current model table.

MAX(employee_id)

Finds latest loaded record.

Real Example

If table contains:

employee_id
1
2
3

New data:

employee_id
4
5

Incremental model loads only:

- 4
 - 5
-

11. Ephemeral Materialization

Syntax

```
{{ config(materialized='ephemeral') }}
```

Features

- No table created
- No view created
- Converts into CTE
- Temporary during query execution

Use Cases

- reusable logic
 - intermediate transformations
-

12. Seeds in DBT

What are Seeds?

Seeds are CSV files loaded into database using DBT.

Stored inside:

```
seeds/
```

Run Seeds

```
dbt seed
```

Purpose

- small datasets
- lookup tables

- sample datasets
 - static reference data
-

13. Sources in DBT

What are Sources?

Sources represent raw existing tables in database.

Defined in:

```
sources:  
  - name: employee_source  
  
  tables:  
    - name: employee_raw_data
```

Access Source

```
{{ source('employee_source', 'employee_raw_data') }}
```

14. ref() Function

Purpose

Used to reference another DBT model.

Syntax

```
{{ ref('final_employee') }}
```

Benefits

- automatic dependency management
 - DAG creation
 - lineage tracking
 - execution order handling
-

15. Staging Layer

Purpose

Initial cleaning and standardization layer.

Usually contains:

- null handling
 - renaming columns
 - type conversion
 - duplicate removal
 - basic transformations
-

Example

```
SELECT
  employee_id,
  UPPER(gender) AS gender,
  CAST(salary AS INTEGER) AS salary

FROM employee_raw_data
```

16. Marts Layer

Purpose

Business-ready analytical layer.

Contains:

- KPIs

- aggregations
- reporting tables
- dashboards tables

17. Snapshots in DBT

Purpose

Track historical changes in data.

Implements Slowly Changing Dimension (SCD Type 2).

Snapshot Syntax

```
{{% snapshot employee_snapshot %}}

{{
  config(
    target_schema='snapshots',
    unique_key='employee_id',
    strategy='check',
    check_cols='all'
  )
}}

SELECT *
FROM {{ ref('employee_cleaned_data') }}

{{% endsnapshot %}}
```

Snapshot Strategies

Strategy	Description
check	compares columns
timestamp	uses updated_at column

Snapshot Columns Created by DBT

Column	Purpose
dbt_scd_id	unique snapshot id
dbt_valid_from	row start date
dbt_valid_to	row end date
dbt_updated_at	update timestamp

18. Tests in DBT

Purpose

Validate data quality.

Generic Tests

unique

Checks duplicate values.

```
- unique
```

not_null

Checks null values.

```
- not_null
```

Example

```
version: 2
```

```
models:
  - name: final_employee

  columns:
    - name: employee_id

    tests:
      - unique
      - not_null
```

Run Tests

```
dbt test
```

19. Macros in DBT

Purpose

Reusable SQL logic.

Stored inside:

```
macros/
```

Example Macro

```
{% macro calculate_bonus(salary) %}

salary * 0.10

{% endmacro %}
```

Use Macro

```
SELECT
    employee_name,
    {{ calculate_bonus('salary') }} AS bonus

FROM final_employee
```

20. analyses Folder

Purpose

Contains analytical SQL queries.

Queries here are NOT executed automatically.

Used for:

- business analysis
- ad hoc queries
- reporting queries

21. tests Folder

Purpose

Contains custom SQL tests.

22. snapshots Folder

Contains snapshot SQL files.

Used for historical tracking.

23. target Folder

Auto-generated by DBT.

Contains:

- compiled SQL
 - manifests
 - catalog
 - documentation files
-

24. logs Folder

Stores execution logs.

Useful for debugging.

25. dbt_project.yml

Main configuration file.

Defines:

- project name
 - model paths
 - materializations
 - configurations
-

26. profiles.yml

Contains database connection details.

Example:

```
employee_project:
  target: dev

  outputs:
```

```
dev:
  type: postgres
  host: localhost
  user: postgres
  password: password
  port: 5432
  dbname: postgres
  schema: public
  threads: 1
```

27. DBT Commands

Initialize Project

```
dbt init project_name
```

Run Models

```
dbt run
```

Run Specific Model

```
dbt run -s model_name
```

Run Seeds

```
dbt seed
```

Run Tests

```
dbt test
```

Run Snapshots

```
dbt snapshot
```

Run Full Pipeline

```
dbt build
```

Generate Documentation

```
dbt docs generate
```

Serve Documentation

```
dbt docs serve
```

28. dbt build Execution Order

When running:

```
dbt build
```

DBT executes:

1. seeds
2. models

- 3. snapshots
- 4. tests

based on dependency order.

29. DAG in DBT

What is DAG?

Directed Acyclic Graph.

Represents model dependencies.

Example:

Raw Data ↓ Staging Model ↓ Final Model ↓ Summary Model

30. Why DBT is Popular

- SQL-based
 - Easy learning curve
 - Fast development
 - Strong documentation
 - Cloud integration
 - Modular transformations
 - Analytics engineering support
 - Excellent lineage tracking
-

31. Real-World DBT Workflow

Step 1

Raw data loaded into warehouse.

Step 2

DBT cleans raw data.

Step 3

DBT creates staging models.

Step 4

DBT creates business-ready tables.

Step 5

Tests validate data quality.

Step 6

Snapshots track history.

Step 7

Dashboards use final models.

32. Difference Between View and Table

Feature	View	Table
Stores Data	No	Yes
Storage	Low	High
Query Speed	Slower	Faster
Refresh	Automatic	Manual rebuild
Use Case	staging	final models

33. Why Incremental Models Use Clean Data

Incremental models usually use cleaned/final tables because:

- avoids bad raw data
- maintains consistency
- improves data quality
- prevents duplicate dirty records

34. Why Tests Can Fail But Data Still Inserts

DBT tests run AFTER models execute.

So: 1. model created 2. data inserted 3. test runs 4. test fails

Data remains unless manually fixed.

35. Best Practices in DBT

- use staging layer
 - keep transformations modular
 - use ref()
 - use tests
 - use snapshots for history
 - use incremental for large tables
 - document models
 - version control with GitHub
-

36. GitHub Deployment

Steps

```
git init
git add .
git commit -m "Initial commit"
git remote add origin REPO_URL
git push -u origin main
```

37. Future Learning Path

DBT ↓ Advanced SQL ↓ PySpark ↓ Airflow ↓ Docker ↓ Snowflake / BigQuery ↓ AWS / GCP / Azure

38. Final Summary

DBT is a powerful transformation tool used in modern Data Engineering and Analytics Engineering pipelines.

It helps:

- transform raw data
- organize SQL workflows
- validate data quality
- track historical changes
- create modular pipelines
- improve analytics workflow

DBT is widely used with:

- Snowflake
- BigQuery
- Redshift
- Databricks
- PostgreSQL

and integrates with:

- Airflow
- GitHub
- CI/CD pipelines
- BI tools.